Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Midrand

Faculty: IT

Group: 1

Lecturer's Name: Mr. Mavetera. T.

Student Full Name: Hamzah Suleman

Student Number: EDUV3475589

Indicate	Yes	No
Plagiarism report attached	yes	

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I also acknowledge that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Eduvos Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature H.S	Date 25 September 2026

Lecturer's Comments:

Marks Awarded:	%

Signature	Date

Eduvos (Pty) Ltd. (formerly Pearson Institute of Higher Education) is registered with the Department of Higher Education and Training as a private higher education institution under the Higher Education Act, 101, of 1997. Registration Certificate number: 2001/HE07/008

Table of Contents

Question 1....3

1.1 Load the data and denoise the text....3

1.2 Identify relevant columns for feature engineering....6

1.3 Baseline model, performance report, and comment....8

Refernces used to help answer this Question....10

Question 2.1....11

Question 2.2....13

Question 2.3....15

References Used to help answer question 2....16

Question 3.1....17

Question 3.2....18

Question 3.3....21

Question 3.4....24

Question 4.1....29

Question 4.2....34

Question 4.3....36

References....36

Question 1

1.1 Load the data and denoise the text

```
# Imports
import re
import string
import unicodedata
import pandas as pd
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

# Download the NLTK resources we need (quietly)
for pkgName in ["stopwords", "wordnet", "omw-1.4"]:
    nltk.download(pkgName, quiet=True)

# Load the dataset
# Try a normal JSON array first, fall back to JSON-lines if that fails
try:
    df = pd.read_json("../data/research_abstracts.json")
except ValueError:
    df = pd.read_json("../data/research_abstracts.json", lines=True)

print(df.shape)
df.info()
print(df.head())
```

```
(1138, 3)
<class 'pandas.core.frame.DataFrame'>
Index: 1138 entries, 0 to 1137
Data columns (total 3 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   DOI       1134 non-null   object
 1   Abstract  1138 non-null   object
 2   Label     1138 non-null   object
dtypes: object(3)
memory usage: 35.6+ KB
                          DOI  \
0        10.1155/2021/1071145
1  10.1007/s43546-022-00384-2
2           10.1002/for.2667
3  10.1007/s42973-021-00091-x
4   10.1007/s40258-012-0003-z

                                            Abstract      Label
0  Forward-looking forecasting of the inflation r...         ai
1  Our study examined the disaggregation of infla...         ai
2  This paper develops a dynamic factor model tha...  classical
3  In this paper, I use a simple SIR Macro model ...  classical
4  BackgroundGame theory is useful for identifyin...  classical
```

```
# Quick data quality check
print("Missing values:\n", df.isna().sum())
print("\nDuplicate DOIs:", df["DOI"].duplicated().sum())
print("Duplicate abstracts:", df["Abstract"].duplicated().sum())
print("\nLabel counts:\n", df["Label"].value_counts())
```

```
Missing values:
 DOI         4
Abstract    0
Label       0
dtype: int64

Duplicate DOIs: 3
Duplicate abstracts: 13

Label counts:
 Label
classical    900
```

```
ai           238
Name: count, dtype: int64

# Check that duplicate abstracts don't have conflicting labels
dupRows = df[df["Abstract"].duplicated(keep=False)]
conflictCount = (dupRows.groupby("Abstract")["Label"].nunique() > 1).sum()
print("Duplicate abstracts with conflicting labels:", conflictCount)

Duplicate abstracts with conflicting labels: 0

# Denoising / text cleaning

# Remove duplicate abstracts (none had conflicting labels, so this is safe)
df = df.drop_duplicates(subset="Abstract").reset_index(drop=True)

stopWords = set(stopwords.words("english"))
lemmatizer = WordNetLemmatizer()

def cleanText(text):
    # Normalise unicode: é -> e, non-breaking space -> space
    text = unicodedata.normalize("NFKD", text)
    # Drop leftover non-ASCII symbols (smart quotes, ©, etc.)
    text = text.encode("ascii", "ignore").decode()
    # Fix section headers glued to the text, e.g. "BackgroundGame" -> "Background Game"
    text = re.sub(r"\b(Background|Objectives?|Methods?|Results|Conclusions?|Purpose)(?=[A-Z])", r"\1 ",
text)
    # Remove publisher boilerplate sentences
    text = re.sub(r"[^.]*(download this paper|www\.nber\.org)[^.]*\.", " ", text, flags=re.I)
    # Remove URLs
    text = re.sub(r"http\S+|www\.\S+", " ", text)
    # Remove citation markers like [1] or [2,3]
    text = re.sub(r"\[\d+(?:[,\-]\s*\d+)*\]", " ", text)
    # Remove copyright lines
    text = re.sub(r"copyright\s*\d{4}[^.]*\.", " ", text, flags=re.I)
    # Lowercase
    text = text.lower()
    # Remove numbers
    text = re.sub(r"\d+", " ", text)
    # Replace punctuation with spaces
    text = text.translate(str.maketrans(string.punctuation, " " * len(string.punctuation)))
    # Remove stopwords and very short tokens, then lemmatize
    return " ".join(lemmatizer.lemmatize(word) for word in text.split()
                    if word not in stopWords and len(word) > 2)

df["cleanText"] = df["Abstract"].apply(cleanText)

# Drop any rows that ended up empty after cleaning
df = df[df["cleanText"].str.len() > 0].reset_index(drop=True)

# Before vs after check
print(df.shape)
print("BEFORE:", df["Abstract"].iloc[0][:300])
print("AFTER: ", df["cleanText"].iloc[0][:300])
print("Avg words before:", df["Abstract"].str.split().str.len().mean())
print("Avg words after:", df["cleanText"].str.split().str.len().mean())

(1125, 4)
BEFORE: Forward-looking forecasting of the inflation rate could help the central bank and other
government departments to better use monetary policy to stabilize prices and prevent the impact of
inflation on market entities, especially for low- and middle-income groups. It can also help financial
institutio
AFTER:  forward looking forecasting inflation rate could help central bank government department better
use monetary policy stabilize price prevent impact inflation market entity especially low middle income
group also help financial institution investor better make investment decision sense forecast inflat
Avg words before: 171.59466666666665
Avg words after: 104.25866666666667
```

1.1 Answer – denoising techniques applied:

- Duplicate abstract removal (13 duplicates, none with conflicting labels)
- Unicode normalisation (accents, non-breaking spaces, smart quotes)

- Removal of boilerplate (e.g. "download this paper at www.nber.org"), URLs, citation markers (`[1]`) and copyright lines
- Fixing section headers glued to text (e.g. `BackgroundGame`)
- Lowercasing, number removal and punctuation removal
- Stopword removal and removal of very short tokens
- Lemmatization

Result: 1138 rows reduced to 1125 and average abstract length reduced from about 172 to about 104 words.

1.2 Identify relevant columns for feature engineering

```
# Check the columns and class balance
print(df.columns.tolist())
print(df["Label"].value_counts())
```

```
['DOI', 'Abstract', 'Label', 'cleanText']
Label
classical    890
ai           235
Name: count, dtype: int64
```

```
# Feature modelling: TF-IDF + chi-square
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.feature_selection import chi2
import numpy as np

# Encode the target: ai = 1, classical = 0
df["labelEnc"] = (df["Label"] == "ai").astype(int)

# Turn the cleaned text into TF-IDF features (unigrams + bigrams)
tfidfVec = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_df=0.9, sublinear_tf=True)
xAll = tfidfVec.fit_transform(df["cleanText"])
yAll = df["labelEnc"]
print("TF-IDF matrix shape:", xAll.shape)

# Chi-square test: which terms best separate ai from classical?
chiScores, pValues = chi2(xAll, yAll)
termNames = np.array(tfidfVec.get_feature_names_out())
topIdx = np.argsort(chiScores)[::-1][:20]
for term, score in zip(termNames[topIdx], chiScores[topIdx]):
    print(f"{term:30s} {score:.2f}")
print("Terms with p < 0.05:", (pValues < 0.05).sum(), "of", len(termNames))
```

```
TF-IDF matrix shape: (1125, 8190)
machine                        22.13
neural                         20.76
network                        20.35
machine learning               20.34
neural network                 20.31
learning                       19.86
algorithm                      15.99
lstm                           11.00
forest                         9.00
svr                            8.84
ann                            8.69
artificial                     8.62
random forest                  8.58
quantile                       8.24
learning model                 7.49
sentiment                      7.40
artificial neural              7.02
learning algorithm             6.89
mse                            6.84
tree                           6.37
Terms with p < 0.05: 47 of 8190
```

1.2 Answer – column selection:

- `cleanText` (from `Abstract`): **feature source**, the only column with semantic content.
- `Label`: **target** (`ai` = 1, `classical` = 0).
- `DOI`: **dropped**. It is just an identifier with no semantic meaning and could leak publisher information (e.g. `10.1016` = Elsevier).
- `Abstract`: **dropped**, it is the raw version of `cleanText` (redundant).

Feature modelling: TF-IDF (unigrams and bigrams) converts the text to numeric features, and a chi-square test identifies which terms carry signal for the label. The top terms (machine, neural,

network, learning, lstm, forest, svr, ann) are all ML vocabulary, as expected for the `ai` class. Only 47 of 8190 terms are significant at $p < 0.05$, so the signal is concentrated in a small set of terms.

1.3 Baseline model, performance report, and comment

```
# Imports for modelling
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score

# Stratified train/test split (keeps the 79/21 class ratio in both sets)
xTrain, xTest, yTrain, yTest = train_test_split(
    df["cleanText"], df["labelEnc"],
    test_size=0.2, stratify=df["labelEnc"], random_state=42
)

# Baseline model: TF-IDF + Logistic Regression
# TF-IDF sits inside the pipeline so it is only fitted on training data (no leakage)
baselineModel = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_df=0.9, sublinear_tf=True)),
    ("clf", LogisticRegression(class_weight="balanced", max_iter=1000, random_state=42))
])
baselineModel.fit(xTrain, yTrain)

Pipeline(steps=[('tfidf',
                 TfidfVectorizer(max_df=0.9, min_df=3, ngram_range=(1, 2),
                                 sublinear_tf=True)),
                ('clf',
                 LogisticRegression(class_weight='balanced', max_iter=1000,
                                    random_state=42))])

# Learning vs generalization: train vs test
trainPred = baselineModel.predict(xTrain)
testPred = baselineModel.predict(xTest)
print("Train acc:", round(accuracy_score(yTrain, trainPred), 3), "| F1:", round(f1_score(yTrain,
trainPred), 3))
print("Test  acc:", round(accuracy_score(yTest, testPred), 3), "| F1:", round(f1_score(yTest,
testPred), 3))

# 5-fold stratified cross-validation (F1 on the ai class)
cvSplitter = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cvF1 = cross_val_score(baselineModel, xTrain, yTrain, cv=cvSplitter, scoring="f1")
print("5-fold CV F1:", cvF1.round(3), "| mean:", round(cvF1.mean(), 3), "| std:", round(cvF1.std(), 3))

# Full performance report on the test set
print("\nClassification report (test):\n",
      classification_report(yTest, testPred, target_names=["classical", "ai"]))
print("Confusion matrix (test):\n", confusion_matrix(yTest, testPred))

Train acc: 0.984 | F1: 0.964
Test  acc: 0.871 | F1: 0.701
5-fold CV F1: [0.838 0.747 0.73  0.806 0.74 ] | mean: 0.772 | std: 0.042

Classification report (test):
               precision    recall  f1-score   support

   classical       0.93      0.91      0.92       178
          ai       0.68      0.72      0.70        47

    accuracy                           0.87       225
   macro avg       0.80      0.82      0.81       225
weighted avg       0.87      0.87      0.87       225

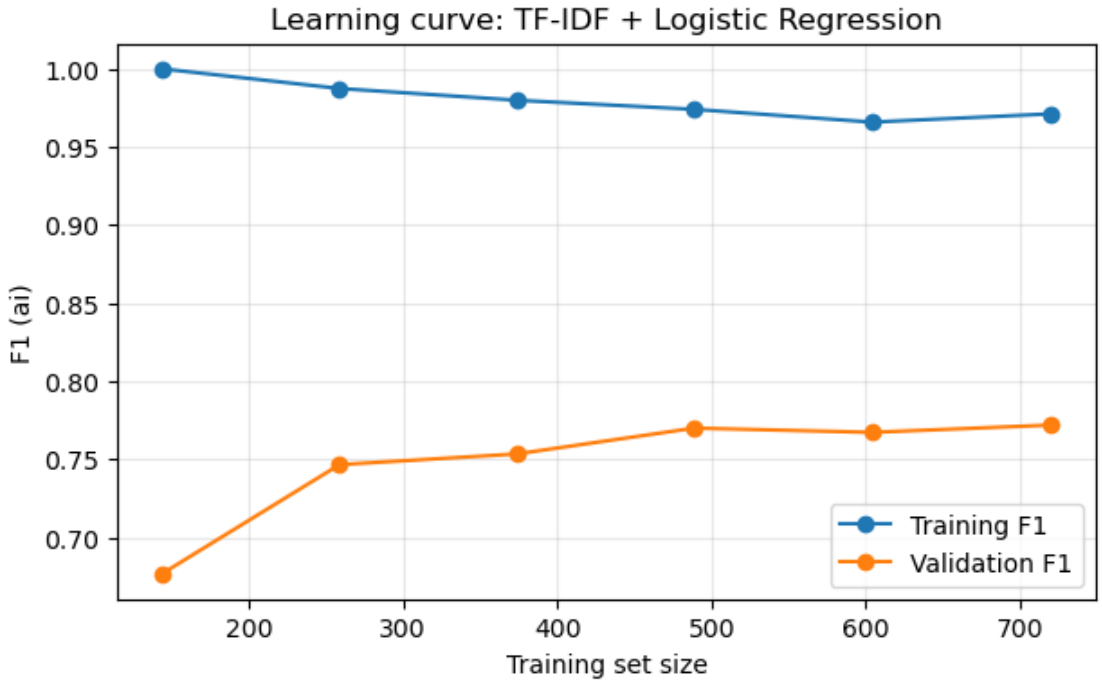
Confusion matrix (test):
 [[162  16]
 [ 13  34]]

# Learning curve: how do train/validation F1 change with more data?
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve

trainSizes, trainScores, valScores = learning_curve(
    baselineModel, xTrain, yTrain, cv=cvSplitter, scoring="f1",
    train_sizes=np.linspace(0.2, 1.0, 6), random_state=42
```

```
)

plt.figure(figsize=(7, 4))
plt.plot(trainSizes, trainScores.mean(axis=1), "o-", label="Training F1")
plt.plot(trainSizes, valScores.mean(axis=1), "o-", label="Validation F1")
plt.xlabel("Training set size")
plt.ylabel("F1 (ai)")
plt.title("Learning curve: TF-IDF + Logistic Regression")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



1.3 Answer – comment on model performance:

	Train	Test
Accuracy	0.984	0.871
F1 (ai)	0.964	0.701

5-fold CV F1 = 0.772 ± 0.042. Test confusion matrix: 162 TN, 16 FP, 13 FN, 34 TP.

- **Learning:** the model fits the training data very well (F1 0.964), so it learns the vocabulary patterns easily.
- **Generalization:** test F1 drops to 0.701 and CV F1 averages 0.772, a gap of roughly 0.2 from training. This indicates **overfitting**, typical for ~8k TF-IDF features on only ~900 training abstracts.
- **Stability:** CV F1 ranges from 0.73 to 0.84, so results vary by split. With only 235 `ai` samples, the single test F1 is a noisy estimate.
- **Class performance:** `classical` performs well (F1 0.92); `ai` is weaker (precision 0.68, recall 0.72). Accuracy (0.87) flatters the model because of the 79/21 imbalance, so F1 on `ai` is the fairer metric.
- **Errors:** 16 false positives and 13 false negatives are roughly balanced, so `class_weight="balanced"` worked as intended.

- **Learning curve:** training F1 stays at ~0.97 while validation F1 rises from ~0.68 to ~0.77 and plateaus. The large persistent gap shows high variance (overfitting), and more data alone would help only marginally.
- **Conclusion:** a reasonable baseline, but it overfits. Stronger regularisation (lower `C`), fewer features, or a different model would be the next improvements.

Refernces used to help answer this Question

I also used AI (DeepSeek) to help me understand the questions and it also helped me formulate a response .I also used it to structure my code more neatly so that it is easier to read and understand.It also helped me interpret the output so that i could answer the following question based on the output of the previous

ApX (n.d.) Practice: preprocessing pipelines. Available at: https://apxml.com/courses/nlp-fundamentals/chapter-1-nlp-text-processing-techniques/practice-preprocessing-pipelines (Accessed: 10 October 2026).

Bird, S., Klein, E. and Loper, E. (2009) Natural language processing with Python. Sebastopol: O'Reilly Media. Available at: https://www.oreilly.com/library/view/natural-language-processing/9780596803346/ch03.html (Accessed: 10 October 2026).

DHRI Curriculum (n.d.) Data cleaning: lemmatizing words. Available at: https://github.com/DHRI-Curriculum/text-analysis/blob/v2.0/sections/10-data-cleaning-lemmatizing-words.md (Accessed: 10 October 2026).

joshtod (2016) PyCon 2016 tutorial. Available at: https://github.com/joshtod/pycon-2016-tutorial (Accessed: 10 October 2026).

Machine Learning Mastery (n.d.) The beginner's guide to natural language processing with Python. Available at: https://machinelearningmastery.com/the-beginners-guide-to-natural-language-processing-with-python/ (Accessed: 10 October 2026).

scikit-learn developers (n.d.a) Common pitfalls and recommended practices. Available at: https://scikit-learn.org/stable/common_pitfalls.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.b) scikit-learn tutorials. Available at: https://scikit-learn.org/stable/tutorial (Accessed: 10 October 2026).

scikit-learn developers (n.d.c) sklearn.feature_extraction.text.TfidfVectorizer. Available at: https://scikit-learn.org/dev/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.d) sklearn.feature_selection.chi2. Available at: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.chi2.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.e) Validation curves: plotting scores to evaluate models. Available at: https://scikit-learn.org/dev/modules/learning_curve.html (Accessed: 10 October 2026).

Question 2.1

```
from sklearn.naive_bayes import ComplementNB
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# Same TF-IDF settings as the baseline, rebuilt for each model
def makeTfidf():
    return TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_df=0.9, sublinear_tf=True)

# The four models to compare
modelDict = {
    "Complement NB": ComplementNB(),
    "Linear SVM": LinearSVC(class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced", random_state=42,
n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
}

resultList = []

# Baseline first, so every model is compared against it
baseTrainPred = baselineModel.predict(xTrain)
baseTestPred = baselineModel.predict(xTest)
resultList.append({
    "Model": "Baseline (Logistic Regression)",
    "Train F1": f1_score(yTrain, baseTrainPred),
    "Test F1": f1_score(yTest, baseTestPred),
    "Test Acc": accuracy_score(yTest, baseTestPred),
    "CV F1 mean": cvF1.mean(),
    "CV F1 std": cvF1.std(),
})

# Train and evaluate each new model
for modelName, clfModel in modelDict.items():
    pipe = Pipeline([("tfidf", makeTfidf()), ("clf", clfModel)])
    pipe.fit(xTrain, yTrain)
    trainPred = pipe.predict(xTrain)
    testPred = pipe.predict(xTest)
    cvScores = cross_val_score(pipe, xTrain, yTrain, cv=cvSplitter, scoring="f1")
    resultList.append({
        "Model": modelName,
        "Train F1": f1_score(yTrain, trainPred),
        "Test F1": f1_score(yTest, testPred),
        "Test Acc": accuracy_score(yTest, testPred),
        "CV F1 mean": cvScores.mean(),
        "CV F1 std": cvScores.std(),
    })

# Comparison table, best CV F1 at the top
resultDf = pd.DataFrame(resultList).sort_values("CV F1 mean", ascending=False).round(3)
print(resultDf.to_string(index=False))
```

```
                      Model  Train F1  Test F1  Test Acc  CV F1 mean  CV F1 std
                  Linear SVM     0.997    0.713     0.889       0.817      0.043
Baseline (Logistic Regression)     0.964    0.701     0.871       0.772      0.042
             Gradient Boosting     0.970    0.744     0.902       0.754      0.051
                 Random Forest     1.000    0.648     0.889       0.715      0.037
                 Complement NB     0.857    0.640     0.880       0.699      0.033
```

2.1 Answer – model comparison:

Four models (Complement Naive Bayes, Linear SVM, Random Forest, Gradient Boosting) were trained on the same TF-IDF features and stratified train/test split as the baseline. They were compared using test F1 (ai class), test accuracy and 5-fold cross-validated F1.

Model	Train F1	Test F1	Test Acc	CV F1 mean	CV F1 std
Linear SVM	0.997	0.713	0.889	0.817	0.043

Model	Train F1	Test F1	Test Acc	CV F1 mean	CV F1 std
Baseline (Logistic Regression)	0.964	0.701	0.871	0.772	0.042
Gradient Boosting	0.970	0.744	0.902	0.754	0.051
Random Forest	1.000	0.648	0.889	0.715	0.037
Complement NB	0.857	0.640	0.880	0.699	0.033

- **Linear SVM** performed best overall. It is the only model that beat the baseline on both test F1 (0.713 vs 0.701) and CV F1 (0.817 vs 0.772).
- **Gradient Boosting** had the highest test F1 (0.744) and accuracy (0.902), but its CV F1 (0.754) is only slightly above the baseline. With only 47 `ai` abstracts in the test set, the single test score is a noisy estimate.
- **Random Forest** fits the training data perfectly (F1 1.000) but scores below the baseline on F1, which indicates overfitting.
- **Complement Naive Bayes** had the lowest scores. Its simple independence assumptions suit this data less well.
- **Conclusion:** Linear SVM is selected as the best single model because it is the most consistent across cross-validation and the test set.

Question 2.2

```
from sklearn.ensemble import VotingClassifier
from sklearn.calibration import CalibratedClassifierCV

# Ensemble: three different models vote by averaging their predicted probabilities
ensembleClf = VotingClassifier(
    estimators=[
        ("svm", CalibratedClassifierCV(LinearSVC(class_weight="balanced", dual=True, random_state=42),
cv=3)),
        ("gb", GradientBoostingClassifier(random_state=42)),
        ("lr", LogisticRegression(class_weight="balanced", max_iter=1000, random_state=42)),
    ],
    voting="soft",
)
ensembleModel = Pipeline([("tfidf", makeTfidf()), ("clf", ensembleClf)])
ensembleModel.fit(xTrain, yTrain)

# Best single model from 2.1, rebuilt for a fair comparison
svmModel = Pipeline([
    ("tfidf", makeTfidf()),
    ("clf", LinearSVC(class_weight="balanced", dual=True, random_state=42)),
])
svmModel.fit(xTrain, yTrain)

# Compare baseline vs best single model vs ensemble
compareList = []
for modelName, fittedModel in [("Baseline (Logistic Regression)", baselineModel),
                               ("Linear SVM", svmModel),
                               ("Soft-voting ensemble", ensembleModel)]:
    testPred = fittedModel.predict(xTest)
    cvScores = cross_val_score(fittedModel, xTrain, yTrain, cv=cvSplitter, scoring="f1")
    compareList.append({
        "Model": modelName,
        "Test F1": f1_score(yTest, testPred),
        "Test Acc": accuracy_score(yTest, testPred),
        "CV F1 mean": cvScores.mean(),
        "CV F1 std": cvScores.std(),
    })
print(pd.DataFrame(compareList).round(3).to_string(index=False))

# Detailed report for the ensemble
ensemblePred = ensembleModel.predict(xTest)
print("\nClassification report (ensemble, test):\n",
      classification_report(yTest, ensemblePred, target_names=["classical", "ai"]))
print("Confusion matrix (ensemble, test):\n", confusion_matrix(yTest, ensemblePred))
```

```
                        Model  Test F1  Test Acc  CV F1 mean  CV F1 std
Baseline (Logistic Regression)    0.701     0.871       0.772      0.042
                   Linear SVM    0.713     0.889       0.817      0.043
          Soft-voting ensemble    0.795     0.924       0.802      0.032

Classification report (ensemble, test):
               precision    recall  f1-score   support

   classical       0.93      0.98      0.95       178
          ai       0.92      0.70      0.80        47

    accuracy                           0.92       225
   macro avg       0.92      0.84      0.87       225
weighted avg       0.92      0.92      0.92       225

Confusion matrix (ensemble, test):
 [[175   3]
 [ 14  33]]
```

2.2 Answer – additional enhancement approach: soft-voting ensemble

Besides trying different models, I applied **ensemble learning**. Three different models (a calibrated Linear SVM, Gradient Boosting and Logistic Regression) each predict a probability, and the ensemble

averages them ("soft voting"). Models make different kinds of errors, so averaging them cancels out some of the individual mistakes. The ensemble sits in the same pipeline as the TF-IDF step, so it is only fitted on training data (no leakage).

Model	Test F1	Test Acc	CV F1 mean	CV F1 std
Baseline (Logistic Regression)	0.701	0.871	0.772	0.042
Linear SVM	0.713	0.889	0.817	0.043
Soft-voting ensemble	0.795	0.924	0.802	0.032

Ensemble test report: classical F1 = 0.95, ai F1 = 0.80 (precision 0.92, recall 0.70). Confusion matrix: 175 TN, 3 FP, 14 FN, 33 TP.

- **Improvement over the baseline:** test F1 rose from 0.701 to 0.795 and accuracy from 0.871 to 0.924. False positives dropped from 16 to 3, and precision on the `ai` class rose from 0.68 to 0.92, so abstracts labelled `ai` are now far more trustworthy.
- **Improvement over the best single model:** test F1 rose from 0.713 (Linear SVM) to 0.795. The ensemble also has the lowest CV standard deviation (0.032), so its performance varies less between splits.
- **Trade-off:** the ensemble's CV F1 (0.802) is slightly below the Linear SVM's (0.817), but within one standard deviation, so the difference is small. Recall on the `ai` class (0.70) is similar to the baseline (0.72), and 14 `ai` abstracts are still missed.
- **Conclusion:** the ensemble gives the best test performance and the most stable results, so it is selected as the final model for deployment.

Question 2.3

```
import joblib
from sklearn.base import clone

# Retrain the final ensemble on all the cleaned data (the best model from 2.2)
finalModel = clone(ensembleModel)
finalModel.fit(df["cleanText"], df["labelEnc"])

# Save it so the web app can load it
joblib.dump(finalModel, "../deployment/researchlyModel.pkl")
print("Model saved")

# Quick sanity check: the saved model expects cleaned text, so clean it first
sampleText = "Machine learning algorithms and neural networks can analyse large volumes of medical data to improve diagnosis."
samplePred = finalModel.predict([cleanText(sampleText)])[0]
print("Prediction:", "artificial intelligence" if samplePred == 1 else "classical")
```

```
Model saved
Prediction: artificial intelligence
```

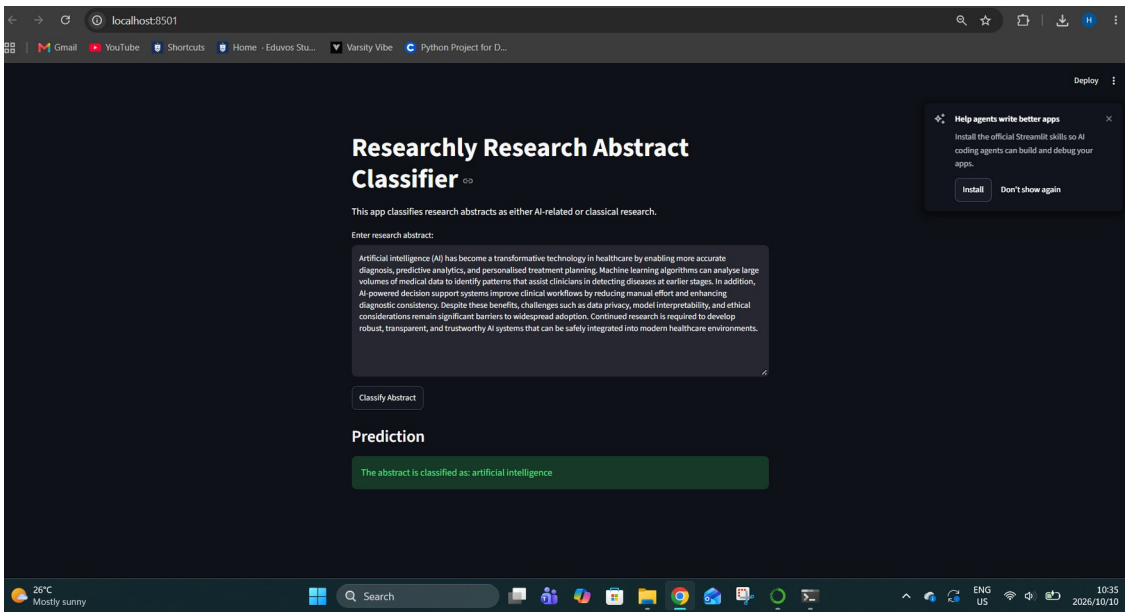


image.png

2.3 Answer – deployment

The best-performing configuration from 2.2 (a soft-voting ensemble of a calibrated Linear SVM, Gradient Boosting and Logistic Regression on TF-IDF features) was retrained on the full cleaned dataset and saved with `joblib` (`researchlyModel.pkl`). A Streamlit web app (`app.py`) loads the saved model. When a user enters an abstract and clicks "Classify Abstract", the app applies the same text-cleaning function used in 1.1 and passes the result to the model. It then displays the prediction as "artificial intelligence" or "classical research". The app is run locally with `streamlit run app.py`.

References Used to help answer question 2

I also used AI (DeepSeek) to help me understand the questions and it also helped me formulate a response .I also used it to structure my code more neatly so that it is easier to read and understand.It also helped me interpret the output so that i could answer the following question based on the output of the previous

Akbari, K. (2024) VotingClassifier in Scikit-Learn: a comprehensive guide. Medium, 19 April. Available at: https://akbarikevin.medium.com/votingclassifier-in-scikit-learn-a-comprehensive-guide-c02bdc55f7f7 (Accessed: 10 October 2026).

scikit-learn developers (n.d.f) sklearn.ensemble. Available at: https://scikit-learn.org/stable/api/sklearn.ensemble.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.g) sklearn.ensemble.VotingClassifier. Available at: https://scikit-learn.org/1.7/modules/generated/sklearn.ensemble.VotingClassifier.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.h) sklearn.svm.LinearSVC. Available at: https://scikit-learn.org/1.4/modules/generated/sklearn.svm.LinearSVC.html (Accessed: 10 October 2026).

scikit-learn developers (n.d.i) User guide. Available at: https://scikit-learn.org/1.4/user_guide.html (Accessed: 10 October 2026).

Streamlit (n.d.) Caching overview. Available at: https://docs.streamlit.io/develop/concepts/architecture/caching (Accessed: 10 October 2026).

Question 3.1

```
import zipfile, os

with zipfile.ZipFile("../data/research_corpus.zip", "r") as z:
    z.extractall("../data")   # creates a research_corpus/ folder

print(sorted(os.listdir("../data/research_corpus")))
```

```
['research_1.txt', 'research_2.txt', 'research_3.txt', 'research_4.txt', 'research_5.txt']
```

```
import pandas as pd

folder = "../data/research_corpus"
records = []

for i, fname in enumerate(sorted(os.listdir(folder)), start=1):
    if not fname.endswith(".txt"):
        continue
    with open(os.path.join(folder, fname), "r", encoding="utf-8", errors="ignore") as f:
        text = f.read()
    records.append({
        "article_id": i,
        "filename": fname,
        "content": text,
        "word_count": len(text.split())
    })

df = pd.DataFrame(records)
print(df.shape)
display(df[["article_id", "filename", "word_count"]])
print("\nFirst 500 characters of article 1:\n")
print(df.loc[0, "content"][:500])
```

```
(5, 4)

   article_id        filename  word_count
0           1  research_1.txt        5602
1           2  research_2.txt        2416
2           3  research_3.txt        3828
3           4  research_4.txt        5922
4           5  research_5.txt        4027

First 500 characters of article 1:

The purposes of this article are to position mixed methods research
(mixed research is a synonym) as the natural complement to traditional qualitative and quantitative
research, to present pragmatism
as offering an attractive philosophical partner for mixed methods research, and to provide a framework
for designing and conducting
mixed methods research. In doing this, we briefly review the paradigm “wars” and incompatibility
thesis, we show some commonalities between quantitative and qualitative
```

```
pd.set_option("display.max_colwidth", 150)
df.head()
```

3.1 Answer: The corpus was extracted from research_corpus.zip, which contains 5 plain-text research articles. Each file was read into a pandas DataFrame with an article ID, filename, full text content and word count.

Question 3.2

```
import re, unicodedata
import nltk, spacy
from collections import Counter

nltk.download("words", quiet=True)
nltk.download("stopwords", quiet=True)
from nltk.corpus import words as nltk_words, stopwords

nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])

# Known words = English dictionary + every whole word that appears in the corpus.
corpus_words = set()
for text in df["content"]:
    corpus_words.update(re.findall(r"(?<= )[a-z]+(?= )", text.lower()))
known_words = set(w.lower() for w in nltk_words.words()) | corpus_words


def fix_line_breaks(text):
    """Re-join words split across lines (e.g. 'pragma\\ntism' -> 'pragmatism')."""
    def join_hyphen(m):
        a, b = m.group(1), m.group(2)
        if (a + b).lower() in known_words:
            return a + b                    # 're-\nsearch' -> 'research'
        return a + "-" + b                  # 'mixed-\nmethods' -> 'mixed-methods'
    text = re.sub(r"([A-Za-z]+)-\n([A-Za-z]+)", join_hyphen, text)

    def join(m):
        a, b = m.group(1).lower(), m.group(2).lower()
        if (a + b) in known_words or (a not in known_words and b not in known_words):
            return m.group(1)
        return m.group(1) + " "
    return re.sub(r"([A-Za-z]+)\n(?=([a-z]+))", join, text)


def remove_references(text):
    """Cut off the reference list if there is one near the end."""
    matches = list(re.finditer(r"\n\s*(references|bibliography|works cited)\s*\n", text, re.I))
    if matches and matches[-1].start() > len(text) * 0.5:
        return text[:matches[-1].start()]
    return text


def light_clean(text):
    """Denoise but keep sentences intact (used for summarisation in 3.3)."""
    text = unicodedata.normalize("NFKD", text)
    text = text.replace("“", '"').replace("”", '"').replace("’", "'").replace("‘", "'")
    text = remove_references(text)
    text = fix_line_breaks(text)
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)              # URLs
    text = re.sub(r"\S+@\S+", " ", text)                            # emails
    text = re.sub(r"\bdoi:?\s*\S+", " ", text, flags=re.I)          # DOIs
    text = re.sub(r"\[[\d,\s\-–]+\]", " ", text)                     # [12], [3-5]
    text = re.sub(r"(?<=[a-z])\.\d{1,2}(?=\s+[A-Z])", ".", text)      # footnotes: 'research.5 We'
    text = re.sub(r"\([^()]*\b(19|20)\d{2}[a-z]?\b[^()]*\)", " ", text)  # (Smith, 2004)
    text = re.sub(r"[^\x00-\x7F]+", " ", text)                       # leftover non-ASCII
    text = re.sub(r"\s+", " ", text)                                 # collapse whitespace
    return text.strip()


stop_words = set(stopwords.words("english")) | {"et", "al", "also", "however", "thus", "may", "one",
"use", "used"}

def deep_clean(text):
    """Full normalisation for topic modelling (3.4): lowercase, letters only, lemmatise, drop
stopwords."""
    text = re.sub(r"[^a-z\s]", " ", text.lower())
    doc = nlp(text)
    tokens = [t.lemma_ for t in doc
              if t.lemma_ not in stop_words and len(t.lemma_) > 2 and t.is_alpha]
    return " ".join(tokens)
```

```
df["clean_text"] = df["content"].apply(light_clean)
df["processed_text"] = df["clean_text"].apply(deep_clean)
df["clean_word_count"] = df["processed_text"].apply(lambda x: len(x.split()))

display(df[["article_id", "filename", "word_count", "clean_word_count"]])
print("\nArticle 1, light clean (first 500 chars):\n", df.loc[0, "clean_text"][:500])
print("\nArticle 1, fully processed (first 300 chars):\n", df.loc[0, "processed_text"][:300])
print("\nTop 15 words across the corpus after cleaning:\n",
      Counter(" ".join(df["processed_text"]).split()).most_common(15))
```

```
   article_id        filename  word_count  clean_word_count
0           1  research_1.txt        5602              2946
1           2  research_2.txt        2416              1162
2           3  research_3.txt        3828              2054
3           4  research_4.txt        5922              3075
4           5  research_5.txt        4027              2057

Article 1, light clean (first 500 chars):
 The purposes of this article are to position mixed methods research (mixed research is a synonym) as
the natural complement to traditional qualitative and quantitative research, to present pragmatism as
offering an attractive philosophical partner for mixed methods research, and to provide a framework for
designing and conducting mixed methods research. In this, we briefly review the paradigm "wars" and
incompatibility thesis, we show some commonalities between quantitative and qualitative resea

Article 1, fully processed (first 300 chars):
 purpose article position mixed method research mixed research synonym natural complement traditional
qualitative quantitative research present pragmatism offer attractive philosophical partner mixed
method research provide framework design conduct mixed method research briefly review paradigm war in

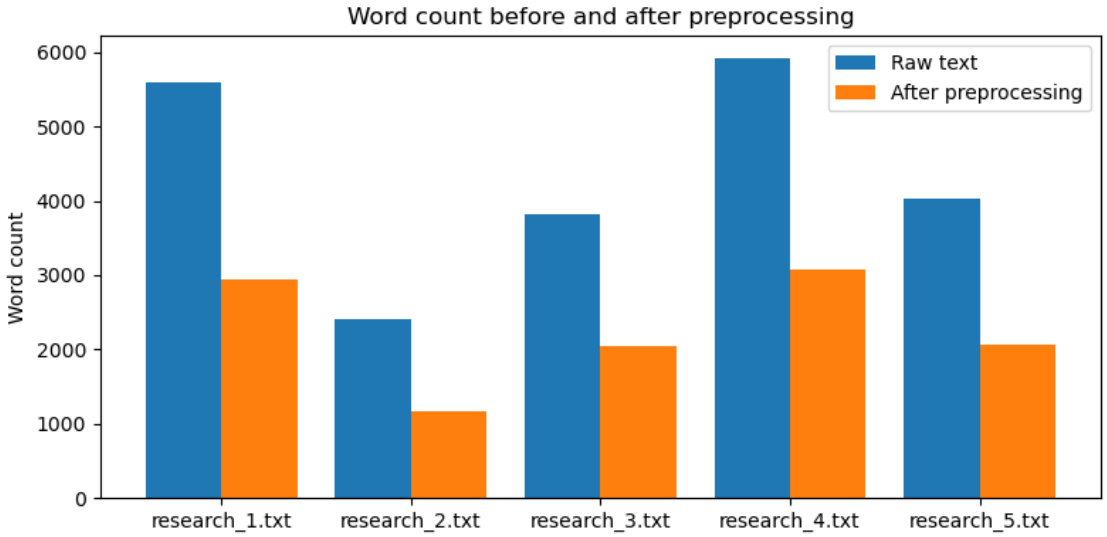
Top 15 words across the corpus after cleaning:
 [('research', 399), ('study', 160), ('researcher', 150), ('method', 128), ('alpha', 119),
('qualitative', 114), ('design', 105), ('datum', 99), ('development', 93), ('triangulation', 84),
('value', 83), ('process', 69), ('instrument', 69), ('science', 64), ('report', 64)]
```

```
import matplotlib.pyplot as plt

# Fix spaCy lemmatising "data" -> "datum"
df["processed_text"] = df["processed_text"].str.replace(r"\bdatum\b", "data", regex=True)
df["clean_word_count"] = df["processed_text"].apply(lambda x: len(x.split()))

# Visualise the effect of preprocessing
x = range(len(df))
plt.figure(figsize=(8, 4))
plt.bar([i - 0.2 for i in x], df["word_count"], width=0.4, label="Raw text")
plt.bar([i + 0.2 for i in x], df["clean_word_count"], width=0.4, label="After preprocessing")
plt.xticks(x, df["filename"])
plt.ylabel("Word count")
plt.title("Word count before and after preprocessing")
plt.legend()
plt.tight_layout()
plt.show()

reduction = (1 - df["clean_word_count"].sum() / df["word_count"].sum()) * 100
print(f"Overall noise reduction: {reduction:.1f}%")
```




```
Overall noise reduction: 48.2%
```

3.2 Answer – denoising techniques applied:

Each article went through a denoising pipeline using spaCy (Honnibal et al., 2020) and NLTK (Bird, Klein and Loper, 2009):

- **Line-break repair:** words split across lines when the text was extracted from PDF (e.g. "pragma / tism") were re-joined using an English dictionary plus the corpus vocabulary
- **Reference lists, URLs, emails, DOIs, in-text citations and footnote numbers** were removed
- **Unicode normalisation** and removal of leftover non-ASCII characters
- **Lowercasing, stopword removal and lemmatisation** (e.g. "studies" → "study"); spaCy's "datum" lemma was mapped back to "data"

Two versions were kept: `clean_text`, which keeps full sentences for extractive summarisation (3.3), and `processed_text`, which is fully normalised for topic modelling (3.4 and 4.1).

Result: Preprocessing reduced the corpus by 48.2%, from 21,795 to 11,294 words. It removed stopwords, citations, punctuation and formatting noise, and kept the meaningful content words.

Question 3.3

```
import networkx as nx
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

article1 = df.loc[0, "clean_text"]

# 1. Split into sentences and drop very short fragments
sentences = re.split(r"(?<=[.!?])\s+(?=[A-Z])", article1)
sentences = [s.strip() for s in sentences if len(s.split()) >= 8]

# 2. Remove duplicate sentences (the source text repeats some passages)
seen, unique = set(), []
for s in sentences:
    key = re.sub(r"\W+", "", s.lower())
    if key not in seen:
        seen.add(key)
        unique.append(s)
print(f"Article 1: {len(sentences)} sentences, {len(sentences) - len(unique)} duplicates removed,
{len(unique)} kept")
sentences = unique

# 3. Clean each sentence the same way as 3.2, then turn it into a TF-IDF vector
clean_sents = [deep_clean(s).replace("datum", "data") for s in sentences]
tfidf = TfidfVectorizer().fit_transform(clean_sents)

# 4. Build a similarity graph and score sentences with PageRank
sim_matrix = cosine_similarity(tfidf)
np.fill_diagonal(sim_matrix, 0)
graph = nx.from_numpy_array(sim_matrix)
scores = nx.pagerank(graph)

# 5. Take the top 7 sentences and put them back in their original order
top_n = 7
top_idx = sorted(sorted(scores, key=scores.get, reverse=True)[:top_n])
summary = " ".join(sentences[i] for i in top_idx)

print(f"\nSUMMARY OF ARTICLE 1 ({len(summary.split())} words, down from {df.loc[0, 'word_count']}):\n")
for i in top_idx:
    print(f"• {sentences[i]}\n")
```

```
Article 1: 150 sentences, 75 duplicates removed, 75 kept

SUMMARY OF ARTICLE 1 (255 words, down from 5602):

• The purposes of this article are to position mixed methods research (mixed research is a synonym) as
the natural complement to traditional qualitative and quantitative research, to present pragmatism as
offering an attractive philosophical partner for mixed methods research, and to provide a framework for
designing and conducting mixed methods research.

• In this, we briefly review the paradigm "wars" and incompatibility thesis, we show some commonalities
between quantitative and qualitative research, we explain the tenets of pragmatism, we explain the
fundamental principle of mixed research and how to apply it, we provide specific sets of designs for
the two major types of mixed methods research (mixed-model designs and mixed-method designs), and,
finally, we explain mixed methods research as following (recursively) an eight-step process.

• Both sets of purists view their paradigms as the ideal for research, and, implicitly if not
explicitly, they advocate the incompatibility thesis , which posits that qualitative and quantitative
research paradigms, including their associated methods, cannot and should not be mixed.

• Our purpose in writing this article is to present mixed methods research as the third research
paradigm in educational research.

• Mixed methods research as the third research paradigm can also help bridge the schism between
quantitative and qualitative research .

• For example, qualitative researchers should be free to use quantitative methods, and quantitative
researchers should be free to use qualitative methods.
```

```
• Mixed methods research should, instead (at this time), use a method and philosophy that attempt to
fit together the insights provided by qualitative and quantitative research into a workable solution.
```

3.3 Answer – summary of article 1 (TextRank):

Article 1 was summarised with **TextRank** (Mihalcea and Tarau, 2004), an extractive, graph-based method. The cleaned article was split into sentences and each sentence was turned into a TF-IDF vector (Pedregosa et al., 2011). Cosine similarity between every pair of sentences was used to build a graph, and PageRank (Page et al., 1999) scored each sentence by how central it is to the rest of the text. The 7 highest-scoring sentences, kept in their original order, form the summary, which reduces the article from 5,602 words to about 255.

While checking the output, 75 of the 150 sentences turned out to be exact duplicates: the article text appears twice in the source file. Duplicates were removed before ranking, because identical sentences are 100% similar to each other and would otherwise push one another to the top.

The summary shows that the article (Johnson and Onwuegbuzie, 2004) presents **mixed methods research as a "third research paradigm"** alongside qualitative and quantitative research. It proposes **pragmatism** as its philosophical basis, rejects the "incompatibility thesis" held by purists on both sides, and offers a framework for designing mixed methods studies.

```python
from wordcloud import WordCloud

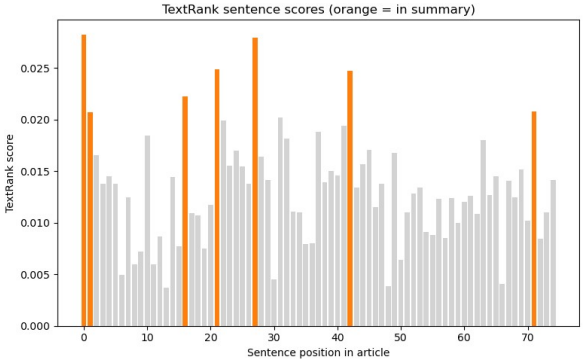
fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# (a) Sentence importance scores: summary sentences highlighted
score_list = [scores[i] for i in range(len(sentences))]
colors = ["tab:orange" if i in top_idx else "lightgrey" for i in range(len(sentences))]
axes[0].bar(range(len(sentences)), score_list, color=colors)
axes[0].set_xlabel("Sentence position in article")
axes[0].set_ylabel("TextRank score")
axes[0].set_title("TextRank sentence scores (orange = in summary)")

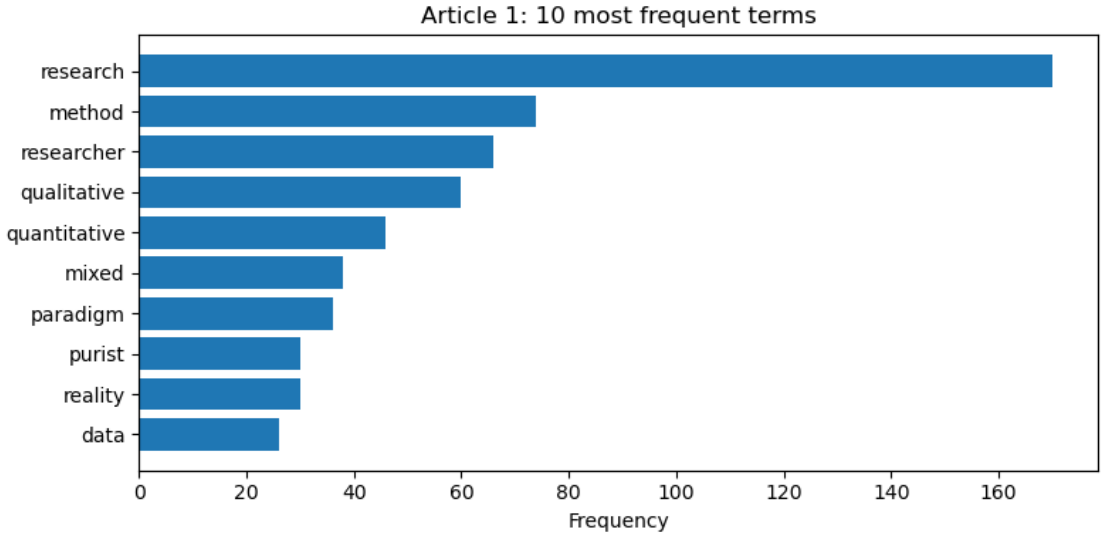
# (b) Word cloud of article 1
wc = WordCloud(width=800, height=400, background_color="white",
               colormap="viridis").generate(df.loc[0, "processed_text"])
axes[1].imshow(wc, interpolation="bilinear")
axes[1].axis("off")
axes[1].set_title("Article 1: key terms")

plt.tight_layout()
plt.show()

# (c) Top 10 terms in article 1
top_terms = Counter(df.loc[0, "processed_text"].split()).most_common(10)
plt.figure(figsize=(8, 4))
plt.barh([w for w, c in top_terms][::-1], [c for w, c in top_terms][::-1], color="tab:blue")
plt.title("Article 1: 10 most frequent terms")
plt.xlabel("Frequency")
plt.tight_layout()
plt.show()
```







3.3 Answer – visualisations: The TextRank score chart shows that most of the top-ranked sentences come from the opening section of the article, where the authors state their aims and argument. The word cloud and frequency chart confirm the article's focus on mixed methods research, qualitative versus quantitative paradigms, and the debate between "purists" on each side.

Question 3.4

```
def dedupe_sentences(text):
    """Remove repeated sentences from an article, keeping the first occurrence."""
    sents = re.split(r"(?<=[.!?])\s+(?=[A-Z])", text)
    seen, kept = set(), []
    for s in sents:
        key = re.sub(r"\W+", "", s.lower())
        if key and key not in seen:
            seen.add(key)
            kept.append(s.strip())
    return " ".join(kept), len(sents) - len(kept), len(sents)

results = df["clean_text"].apply(dedupe_sentences)
df["clean_text"] = results.apply(lambda r: r[0])
df["duplicates_removed"] = results.apply(lambda r: r[1])
df["total_sentences"] = results.apply(lambda r: r[2])

# Re-process the de-duplicated text
df["processed_text"] = df["clean_text"].apply(deep_clean).str.replace(r"\bdatum\b", "data", regex=True)
df["clean_word_count"] = df["processed_text"].apply(lambda x: len(x.split()))

display(df[["article_id", "filename", "total_sentences", "duplicates_removed", "clean_word_count"]])
```

```
   article_id        filename  total_sentences  duplicates_removed  \
0           1  research_1.txt              152                  76
1           2  research_2.txt               57                   0
2           3  research_3.txt              150                   0
3           4  research_4.txt              176                   0
4           5  research_5.txt              161                   0

   clean_word_count
0              1473
1              1162
2              2054
3              3075
4              2057
```

Approach: 3.3 used TextRank, which picks the most important sentences from one article. For 3.4 the question asks for a different technique that summarises the whole group at once, so I used **LDA topic modelling** (Blei, Ng and Jordan, 2003). It finds the main themes across the four articles and the key words that define each theme.

LDA needs many documents to learn from, and four articles are too few. So each article is split into chunks of about five sentences, and every chunk is treated as a small document.

```
from gensim import corpora
from gensim.models import LdaModel, CoherenceModel

# Articles 2–5 only
rest = df[df["article_id"] > 1].reset_index(drop=True)

# Split each article into chunks of 5 sentences; each chunk is one "document" for LDA
chunk_size = 5
docs, doc_article = [], []
for _, row in rest.iterrows():
    sents = re.split(r"(?<=[.!?])\s+(?=[A-Z])", row["clean_text"])
    for i in range(0, len(sents), chunk_size):
        tokens = deep_clean(" ".join(sents[i:i + chunk_size])).replace("datum", "data").split()
        if len(tokens) >= 10:
            docs.append(tokens)
            doc_article.append(row["filename"])

print(f"{len(docs)} text chunks created from {len(rest)} articles")
print(pd.Series(doc_article).value_counts().sort_index())

# Build the dictionary and bag-of-words corpus
dictionary = corpora.Dictionary(docs)
dictionary.filter_extremes(no_below=2, no_above=0.5)   # drop very rare and very common words
```

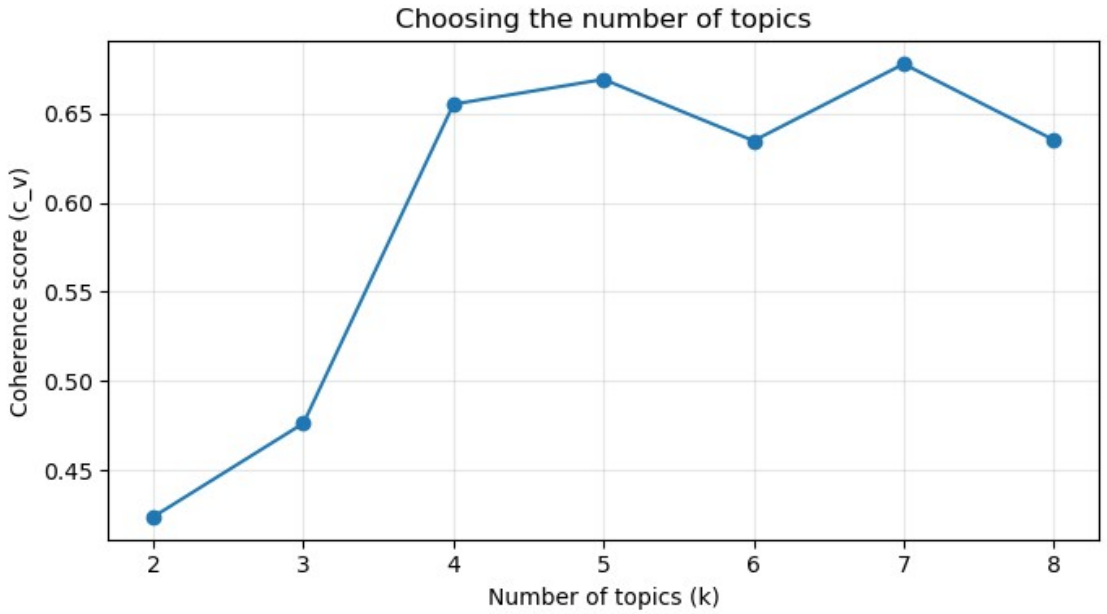
```python
corpus = [dictionary.doc2bow(d) for d in docs]
print(f"\nVocabulary size after filtering: {len(dictionary)}")

# Try 2 to 8 topics and measure coherence (higher = more interpretable topics)
coherence_scores = {}
for k in range(2, 9):
    model = LdaModel(corpus=corpus, id2word=dictionary, num_topics=k,
                     random_state=42, passes=15, alpha="auto")
    cm = CoherenceModel(model=model, texts=docs, dictionary=dictionary,
                        coherence="c_v", processes=1)
    coherence_scores[k] = cm.get_coherence()
    print(f"k = {k}: coherence = {coherence_scores[k]:.4f}")

plt.figure(figsize=(7, 4))
plt.plot(list(coherence_scores.keys()), list(coherence_scores.values()), marker="o")
plt.xlabel("Number of topics (k)")
plt.ylabel("Coherence score (c_v)")
plt.title("Choosing the number of topics")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
110 text chunks created from 4 articles
research_2.txt    12
research_3.txt    30
research_4.txt    35
research_5.txt    33
Name: count, dtype: int64

Vocabulary size after filtering: 928
k = 2: coherence = 0.4239
k = 3: coherence = 0.4763
k = 4: coherence = 0.6551
k = 5: coherence = 0.6691
k = 6: coherence = 0.6346
k = 7: coherence = 0.6777
k = 8: coherence = 0.6351
```




```python
best_k = 5
lda = LdaModel(corpus=corpus, id2word=dictionary, num_topics=best_k,
               random_state=42, passes=15, alpha="auto")

final_cm = CoherenceModel(model=lda, texts=docs, dictionary=dictionary,
                          coherence="c_v", processes=1)
print(f"Final model coherence: {final_cm.get_coherence():.4f}\n")
```

```python
for t in range(best_k):
    words = ", ".join(w for w, p in lda.show_topic(t, topn=10))
    print(f"Topic {t+1}: {words}")
```

```
Final model coherence: 0.6691

Topic 1: knowledge, instrument, alpha, science, report, scale, measure, cronbach, item, education
Topic 2: researcher, experience, reflexivity, qualitative, position, share, personal, social, change,
address
Topic 3: data, triangulation, method, design, researcher, qualitative, multiple, denzin, student, case
Topic 4: development, design, developmental, process, instructional, evaluation, type, product, field,
term
Topic 5: alpha, value, cronbach, reliability, internal, consistency, report, statistic, paper, science
```

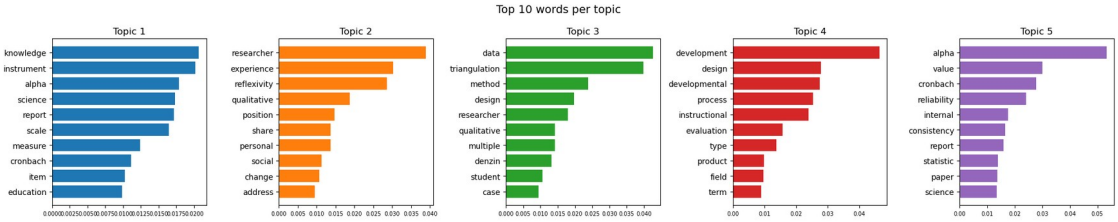
```python
# Topic mix for every chunk, then averaged per article
doc_topics = np.zeros((len(corpus), best_k))
for i, bow in enumerate(corpus):
    for t, p in lda.get_document_topics(bow, minimum_probability=0):
        doc_topics[i, t] = p
topic_cols = [f"Topic {t+1}" for t in range(best_k)]
dt_df = pd.DataFrame(doc_topics, columns=topic_cols)
dt_df["article"] = doc_article
article_topics = dt_df.groupby("article")[topic_cols].mean()

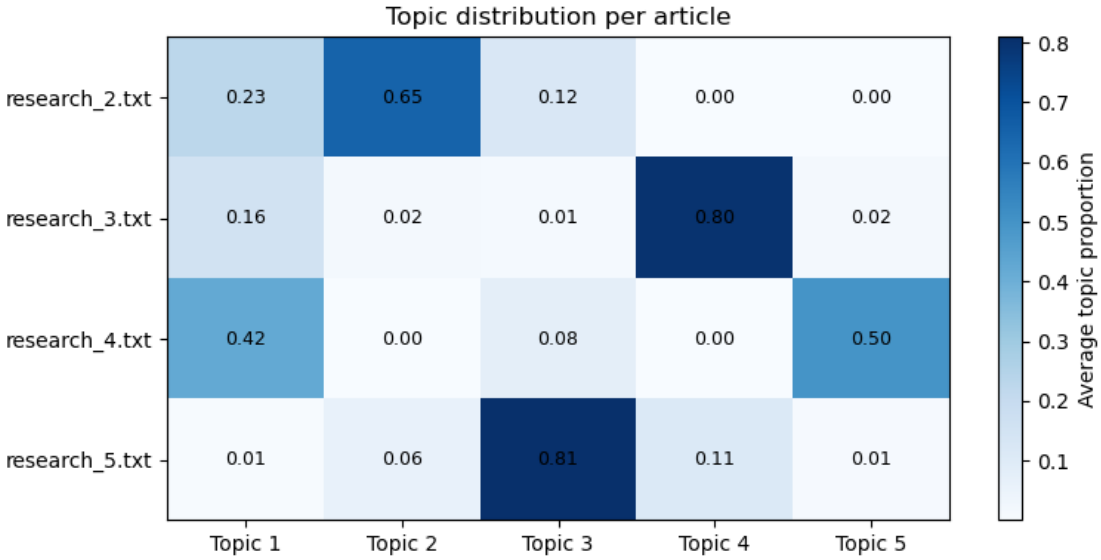
# (a) Top 10 words per topic
fig, axes = plt.subplots(1, best_k, figsize=(4 * best_k, 4))
for t in range(best_k):
    words, probs = zip(*lda.show_topic(t, topn=10))
    axes[t].barh(words[::-1], probs[::-1], color=f"C{t}")
    axes[t].set_title(f"Topic {t+1}")
    axes[t].tick_params(axis="x", labelsize=7)
fig.suptitle("Top 10 words per topic", fontsize=14)
plt.tight_layout()
plt.show()

# (b) Heatmap: how much each article is about each topic
plt.figure(figsize=(8, 4))
plt.imshow(article_topics.values, cmap="Blues", aspect="auto")
plt.colorbar(label="Average topic proportion")
plt.xticks(range(best_k), topic_cols)
plt.yticks(range(len(article_topics)), article_topics.index)
for i in range(article_topics.shape[0]):
    for j in range(best_k):
        plt.text(j, i, f"{article_topics.values[i, j]:.2f}", ha="center", va="center", fontsize=9)
plt.title("Topic distribution per article")
plt.tight_layout()
plt.show()

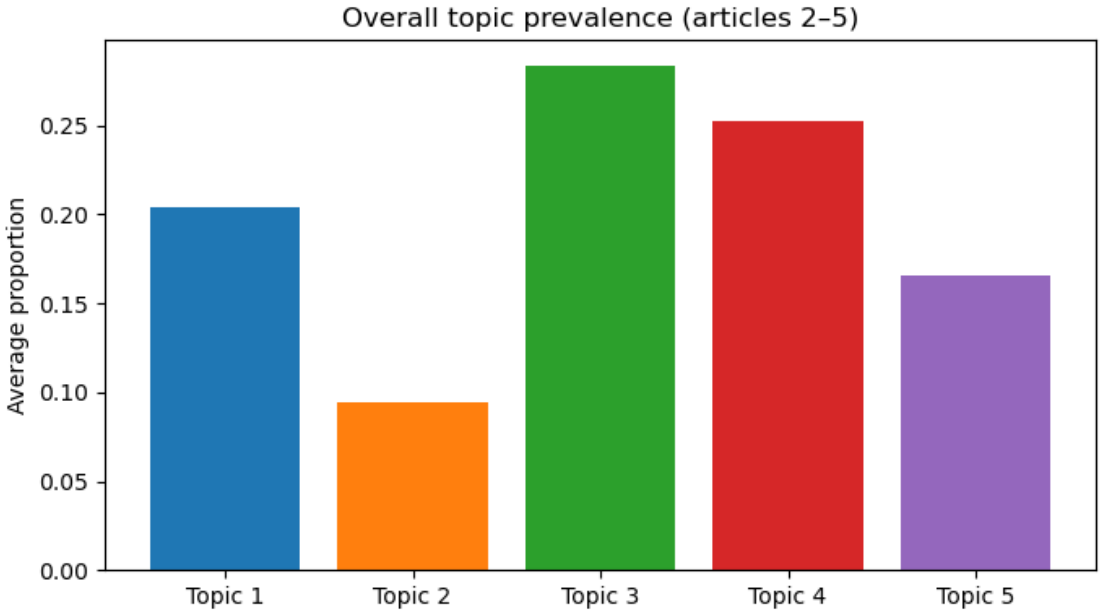
# (c) Word cloud per topic
fig, axes = plt.subplots(1, best_k, figsize=(4 * best_k, 3.5))
for t in range(best_k):
    wc = WordCloud(width=400, height=300, background_color="white", colormap="tab10") \
         .generate_from_frequencies(dict(lda.show_topic(t, topn=30)))
    axes[t].imshow(wc, interpolation="bilinear")
    axes[t].axis("off")
    axes[t].set_title(f"Topic {t+1}")
plt.tight_layout()
plt.show()

# (d) Overall share of each topic across the whole corpus
plt.figure(figsize=(7, 4))
plt.bar(topic_cols, dt_df[topic_cols].mean(), color=[f"C{t}" for t in range(best_k)])
plt.ylabel("Average proportion")
plt.title("Overall topic prevalence (articles 2–5)")
plt.tight_layout()
plt.show()
```









3.4 Answer – aggregate summary of articles 2–5:

For the remaining four articles I used **LDA topic modelling** (Blei, Ng and Jordan, 2003). Instead of picking out sentences like TextRank did in 3.3, LDA looks at which words tend to appear together and groups them into topics. That makes it a good way to summarise several articles at once.

Four articles aren't enough for LDA on their own, so I split each one into chunks of five sentences, which gave 110 small documents. I tested between 2 and 8 topics and picked **5**. Its c_v coherence score (0.669) (Röder, Both and Hinneburg, 2015) was almost as high as the best one (0.678 at 7 topics), and 5 topics are easier to make sense of.

The 5 topics:

1. **Measurement tools in science education**: instruments, scales, items
2. **Reflexivity**: how a researcher's own background and position affect their research
3. **Triangulation**: using several data sources or methods to check findings
4. **Developmental research**: designing and testing instructional products
5. **Reliability**: using and reporting Cronbach's alpha

What the articles are about: The heatmap shows that each article focuses mainly on one topic. Article 2 is about reflexivity, Article 3 about developmental research, Article 4 about measurement tools and Cronbach's alpha, and Article 5 about triangulation. Put together, all four articles deal with the same big question: **how to make research trustworthy**. Each one covers a different way of doing that, which fits well with Article 1's argument for mixed methods research.

Limitations: Topics 1 and 5 overlap a bit because both come from Article 4. With only four articles, the topics mostly line up with one article each instead of spreading across all of them.

Note: The brief describes the corpus as AI research articles, but the topic modelling shows the articles are about research methodology and research quality, not AI. This shows how topic modelling can reveal what a collection is really about, regardless of how it was labelled.

Question 4.1

```
from gensim.models.phrases import Phrases, Phraser

# Split all 5 articles into 5-sentence chunks (each chunk = one document for LDA)
docs_all, doc_article_all = [], []
for _, row in df.iterrows():
    sents = re.split(r"(?<=[.!?])\s+(?=[A-Z])", row["clean_text"])
    for i in range(0, len(sents), 5):
        tokens = deep_clean(" ".join(sents[i:i + 5])).replace("datum", "data").split()
        if len(tokens) >= 10:
            docs_all.append(tokens)
            doc_article_all.append(row["filename"])

# Detect common two-word phrases (bigrams), e.g. "mixed method" -> "mixed_method"
bigram = Phraser(Phrases(docs_all, min_count=3, threshold=10))
docs_all = [bigram[d] for d in docs_all]
found_bigrams = sorted({w for d in docs_all for w in d if "_" in w})
print(f"{len(docs_all)} chunks from {df.shape[0]} articles")
print(f"\nBigrams detected ({len(found_bigrams)}):", found_bigrams[:25])

# Dictionary and corpus
dictionary_all = corpora.Dictionary(docs_all)
dictionary_all.filter_extremes(no_below=2, no_above=0.5)
corpus_all = [dictionary_all.doc2bow(d) for d in docs_all]
print(f"\nVocabulary size: {len(dictionary_all)}")

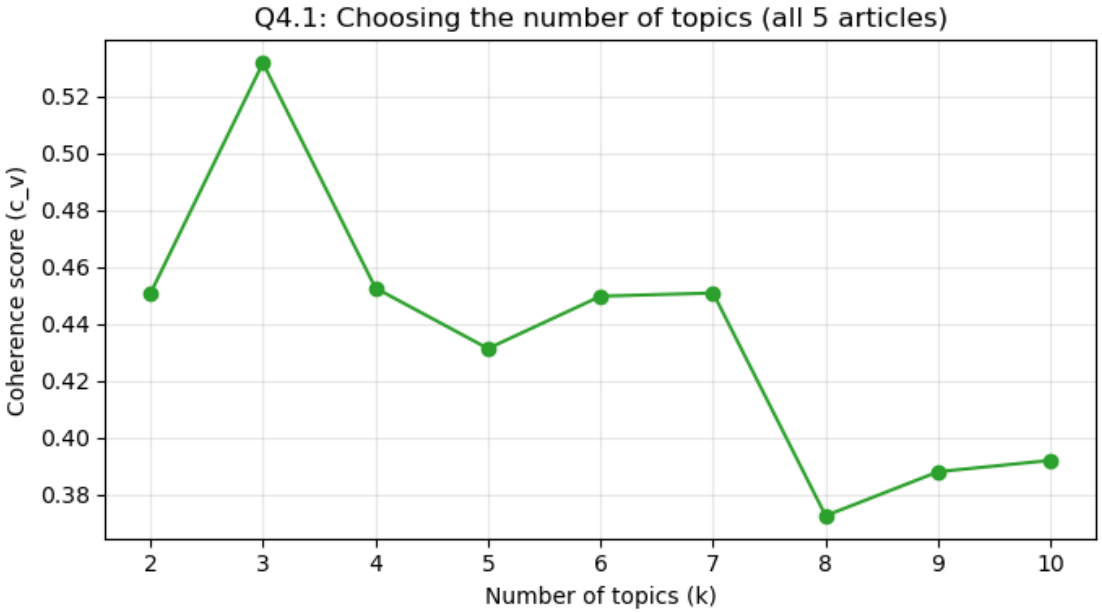
# Test 2–10 topics
coherence_all = {}
for k in range(2, 11):
    m = LdaModel(corpus=corpus_all, id2word=dictionary_all, num_topics=k,
                 random_state=42, passes=15, alpha="auto")
    coherence_all[k] = CoherenceModel(model=m, texts=docs_all, dictionary=dictionary_all,
                                      coherence="c_v", processes=1).get_coherence()
    print(f"k = {k}: coherence = {coherence_all[k]:.4f}")

plt.figure(figsize=(7, 4))
plt.plot(list(coherence_all.keys()), list(coherence_all.values()), marker="o", color="tab:green")
plt.xlabel("Number of topics (k)")
plt.ylabel("Coherence score (c_v)")
plt.title("Q4.1: Choosing the number of topics (all 5 articles)")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
126 chunks from 5 articles

Bigrams detected (63): ['acceptable_value', 'alpha_statistic', 'alpha_value', 'author_report',
'blend_design', 'case_study', 'conceptual_framework', 'construct_validity', 'context_specific',
'cronbach_alpha', 'data_collect', 'data_collection', 'data_saturation', 'data_triangulation',
'design_development', 'development_evaluation', 'development_process', 'developmental_research',
'diagnostic_instrument', 'educational_researcher', 'evaluation_process', 'even_though',
'explicit_reference', 'factor_analysis', 'four_type']

Vocabulary size: 1119
k = 2: coherence = 0.4507
k = 3: coherence = 0.5316
k = 4: coherence = 0.4526
k = 5: coherence = 0.4314
k = 6: coherence = 0.4497
k = 7: coherence = 0.4508
k = 8: coherence = 0.3726
k = 9: coherence = 0.3881
k = 10: coherence = 0.3921
```




```
for k in [3, 5]:
    m = LdaModel(corpus=corpus_all, id2word=dictionary_all, num_topics=k,
                 random_state=42, passes=15, alpha="auto")
    print(f"===== k = {k}  (coherence {coherence_all[k]:.4f}) =====")
    for t in range(k):
        print(f"Topic {t+1}: " + ", ".join(w for w, p in m.show_topic(t, topn=10)))
    print()
```

```
===== k = 3  (coherence 0.5316) =====
Topic 1: researcher, data, triangulation, method, design, denzin, mixed_method, qualitative_research,
example, multiple
Topic 2: process, alpha, report, development, knowledge, reflexivity, cronbach_alpha, value, scale,
reliability
Topic 3: instrument, scale, item, measure, cronbach_alpha, report, reliability, development, test,
consider

===== k = 5  (coherence 0.4314) =====
Topic 1: data, design, field, triangulation, case_study, point, researcher, development, process,
student
Topic 2: alpha, reflexivity, knowledge, report, scale, process, cronbach_alpha, researcher, item,
article
Topic 3: instrument, test, cronbach_alpha, measure, item, report, scale, reliability,
science_education, consider
Topic 4: method, triangulation, denzin, mixed_method, researcher, data, data_triangulation, multiple,
paradigm, design
Topic 5: researcher, design, analysis, evaluation, developmental_research, development, reality,
process, example, important
```

Choice of k: k = 3 had the highest coherence (0.53), but it merged unrelated themes into one topic. k = 5 was chosen because it gave more distinct, interpretable topics and fits the 5-article corpus better.

```
best_k_all = 5
lda_all = LdaModel(corpus=corpus_all, id2word=dictionary_all, num_topics=best_k_all,
                   random_state=42, passes=15, alpha="auto")
topic_cols_all = [f"Topic {t+1}" for t in range(best_k_all)]

# ---- Topic word lists with their probabilities (topic–word distribution) ----
print("TOPIC–WORD DISTRIBUTION (top 10 words and their weights)\n")
for t in range(best_k_all):
    print(f"Topic {t+1}: " + " + ".join(f"{p:.3f}*{w}" for w, p in lda_all.show_topic(t, topn=10)))

# ---- Topic distribution per chunk and per article (document–topic distribution) ----
dt_all = np.zeros((len(corpus_all), best_k_all))
for i, bow in enumerate(corpus_all):
    for t, p in lda_all.get_document_topics(bow, minimum_probability=0):
```

```
        dt_all[i, t] = p
dt_all_df = pd.DataFrame(dt_all, columns=topic_cols_all)
dt_all_df["article"] = doc_article_all
dt_all_df["dominant_topic"] = dt_all_df[topic_cols_all].idxmax(axis=1)
article_mix = dt_all_df.groupby("article")[topic_cols_all].mean()

print("\nDOCUMENT–TOPIC DISTRIBUTION (average per article)\n")
display(article_mix.round(2))

# (a) Top 10 words per topic
fig, axes = plt.subplots(1, best_k_all, figsize=(4.2 * best_k_all, 4))
for t in range(best_k_all):
    words, probs = zip(*lda_all.show_topic(t, topn=10))
    axes[t].barh(words[::-1], probs[::-1], color=f"C{t}")
    axes[t].set_title(f"Topic {t+1}")
    axes[t].tick_params(axis="x", labelsize=7)
fig.suptitle("Q4.1: Top 10 words per topic (all 5 articles)", fontsize=14)
plt.tight_layout()
plt.show()

# (b) Stacked bar: topic mix in each article
article_mix.plot(kind="barh", stacked=True, figsize=(9, 4), color=[f"C{t}" for t in range(best_k_all)])
plt.xlabel("Proportion of article")
plt.title("Q4.1: Topic distribution per article")
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
plt.tight_layout()
plt.show()

# (c) Overall topic distribution: how many chunks each topic dominates
counts = dt_all_df["dominant_topic"].value_counts().reindex(topic_cols_all, fill_value=0)
plt.figure(figsize=(7, 4))
plt.bar(counts.index, counts.values, color=[f"C{t}" for t in range(best_k_all)])
plt.ylabel("Number of text chunks")
plt.title("Q4.1: Dominant topic across all chunks")
plt.tight_layout()
plt.show()

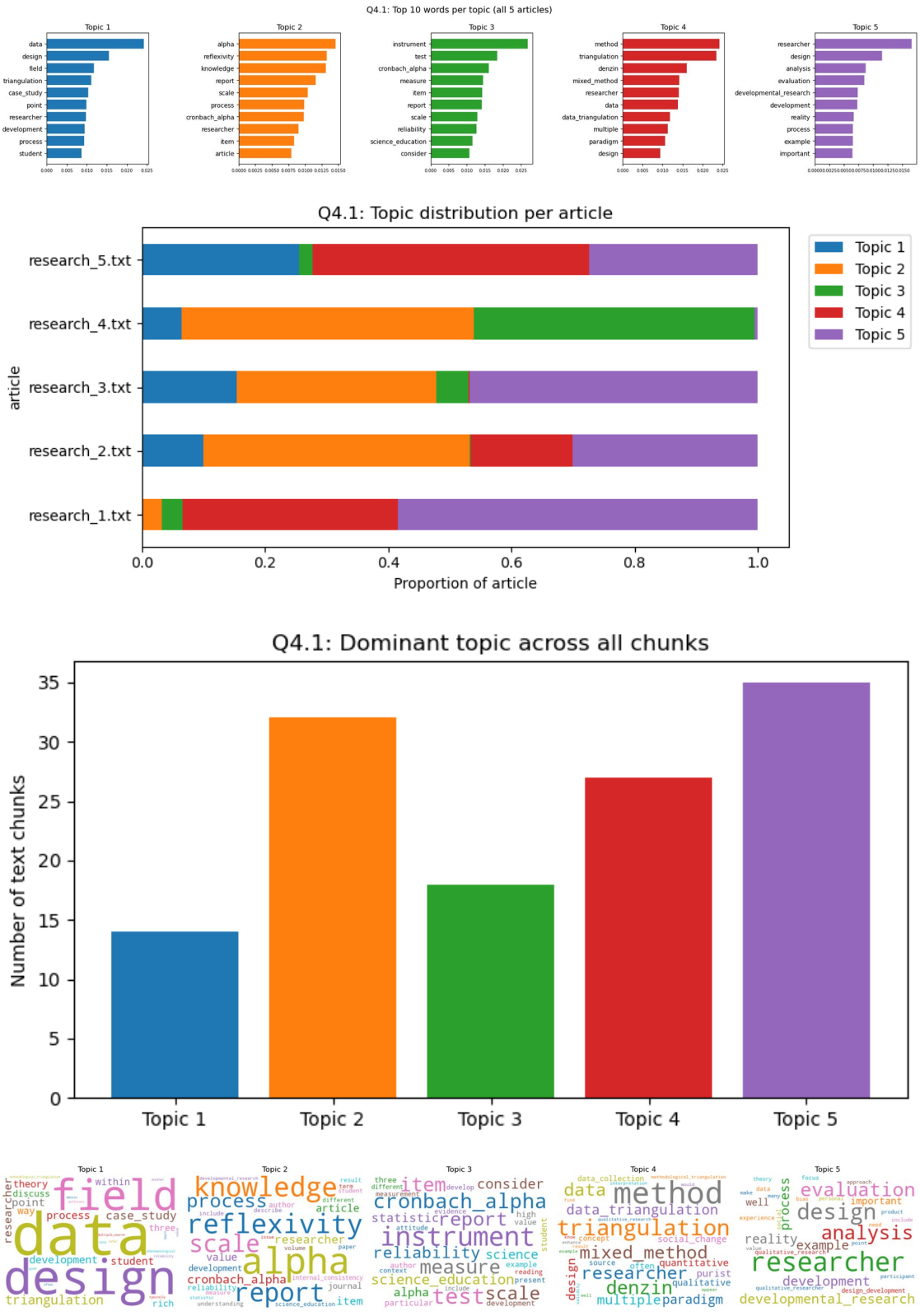
# (d) Word clouds per topic
fig, axes = plt.subplots(1, best_k_all, figsize=(4.2 * best_k_all, 3.5))
for t in range(best_k_all):
    wc = WordCloud(width=400, height=300, background_color="white", colormap="tab10") \
         .generate_from_frequencies(dict(lda_all.show_topic(t, topn=30)))
    axes[t].imshow(wc, interpolation="bilinear")
    axes[t].axis("off")
    axes[t].set_title(f"Topic {t+1}")
plt.tight_layout()
plt.show()

TOPIC–WORD DISTRIBUTION (top 10 words and their weights)

Topic 1: 0.024*data + 0.015*design + 0.012*field + 0.011*triangulation + 0.010*case_study + 0.010*point
+ 0.010*researcher + 0.009*development + 0.009*process + 0.009*student
Topic 2: 0.015*alpha + 0.013*reflexivity + 0.013*knowledge + 0.012*report + 0.010*scale + 0.010*process
+ 0.010*cronbach_alpha + 0.009*researcher + 0.008*item + 0.008*article
Topic 3: 0.027*instrument + 0.018*test + 0.016*cronbach_alpha + 0.014*measure + 0.014*item +
0.014*report + 0.013*scale + 0.013*reliability + 0.012*science_education + 0.011*consider
Topic 4: 0.024*method + 0.023*triangulation + 0.016*denzin + 0.014*mixed_method + 0.014*researcher +
0.014*data + 0.012*data_triangulation + 0.011*multiple + 0.011*paradigm + 0.009*design
Topic 5: 0.017*researcher + 0.012*design + 0.009*analysis + 0.009*evaluation +
0.007*developmental_research + 0.007*development + 0.007*reality + 0.007*process + 0.006*example +
0.006*important

DOCUMENT–TOPIC DISTRIBUTION (average per article)

                Topic 1  Topic 2  Topic 3  Topic 4  Topic 5
article
research_1.txt     0.00     0.03     0.04     0.35     0.58
research_2.txt     0.10     0.43     0.00     0.17     0.30
research_3.txt     0.15     0.32     0.05     0.00     0.47
research_4.txt     0.06     0.47     0.46     0.00     0.01
research_5.txt     0.25     0.00     0.02     0.45     0.27
```



4.1 Answer – LDA topic modelling (all 5 articles):

I ran LDA (Blei, Ng and Jordan, 2003) on all five articles, split into 126 chunks of five sentences each. This time I also added **bigrams** (Mikolov et al., 2013), so common two-word phrases like

cronbach_alpha, *mixed_method* and *case_study* are treated as single terms. That makes the topics more specific.

I tested 2 to 10 topics. k = 3 had the highest coherence (0.53), but it merged unrelated ideas into one topic. I chose **k = 5** because the topics were clearer and easier to interpret, and it suits a five-article corpus.

The 5 topics:

1. **Fieldwork and case studies**: data, field, case_study, student
2. **Reporting and reflection**: alpha, reflexivity, knowledge, report (a mixed topic)
3. **Measurement and reliability**: instrument, test, cronbach_alpha, reliability
4. **Mixed methods and triangulation**: method, triangulation, Denzin, mixed_method, paradigm
5. **Research design and development**: design, analysis, evaluation, developmental_research

What the charts show: The topic-word weights show which words define each topic. The stacked bar chart shows each article as a mix of topics. Article 4 is mostly about measurement and reliability (Topics 2 and 3), and Article 1 is mostly research design and mixed methods (Topics 4 and 5). Unlike in 3.4, several topics are shared between articles. For example, Topic 4 connects Article 1's mixed methods argument with Article 5's discussion of triangulation. Topic 5 is the most common topic overall, which makes sense because research design comes up in every article.

Limitation: Topic 2 mixes reflexivity with reliability reporting. A larger corpus would probably separate these into their own topics.

Question 4.2

```
import pyLDAvis
import pyLDAvis.gensim_models as gensimvis

pyLDAvis.enable_notebook()
vis = gensimvis.prepare(lda_all, corpus_all, dictionary_all, sort_topics=False)

# Save a standalone HTML copy (useful for the GitHub repo in 4.3)
pyLDAvis.save_html(vis, "lda_intertopic_map.html")

vis
```

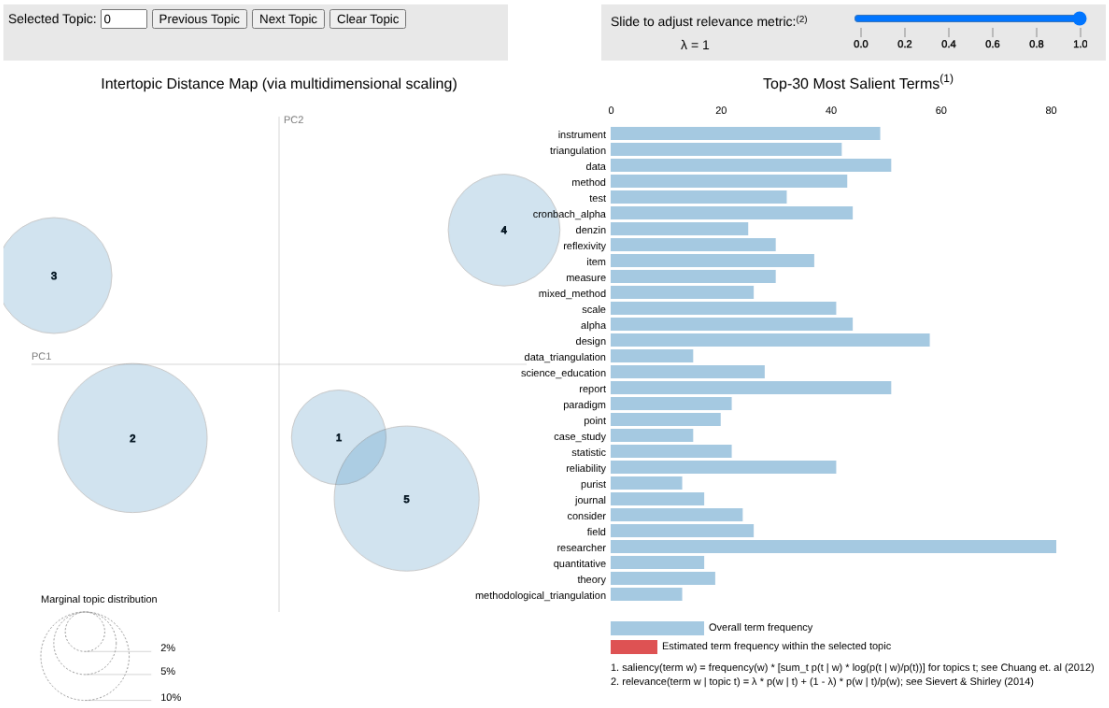
4.2 Answer – intertopic distance map (pyLDAvis):

The interactive map above (Sievert and Shirley, 2014) shows the 5 topics from 4.1. Each circle is a topic: its **size** shows how common the topic is, and the **distance** between circles shows how different the topics are. Clicking a circle shows its most salient words on the right, and the λ slider changes the ranking from the most frequent words (λ = 1) to the words most specific to that topic (lower λ).

What the map shows:

- **Topics 2, 3 and 4 are well separated**, so reflection and reporting, measurement and reliability, and mixed methods and triangulation are clearly distinct themes.
- **Topics 1 and 5 overlap**, because fieldwork and case studies and research design share a lot of vocabulary (data, design, researcher, process). This fits the 4.1 results, where Topic 5 was the most common topic and appeared across most articles.
- The **most salient terms** across the whole corpus are *researcher, design, data, report, instrument, cronbach_alpha* and *triangulation*. Together they confirm that the corpus is about research methodology and research quality.

An interactive HTML version of the map is saved as `lda_intertopic_map.html`.



Intertopic Distance Map (pyLDAvis) for the 5-topic LDA model

Question 4.3

4.3 Answer – GitHub repository:

All project files are available at: **https://github.com/Hamzah5566/ITNPA-Researchly-EDUV3475589**

Repository structure:

- `README.md`: project overview and instructions
- `docs/`: project documentation for Questions 1–4
- `data/`: project datasets (`research_abstracts.json` and `research_corpus.zip`)
- `notebooks/`: Jupyter notebooks for Questions 1–2 and Questions 3–4
- `deployment/`: the Streamlit app, saved model, requirements and README for the design from Questions 1 & 2

References

Bird, S., Klein, E. and Loper, E. (2009) *Natural Language Processing with Python*. Sebastopol, CA: O'Reilly Media. Available at: https://www.nltk.org/book/ (Accessed: 10 October 2026).

Blei, D.M., Ng, A.Y. and Jordan, M.I. (2003) 'Latent Dirichlet allocation', *Journal of Machine Learning Research*, 3, pp. 993–1022. Available at: https://www.jmlr.org/papers/volume3/blei03a/blei03a.pdf (Accessed: 10 October 2026).

Chuang, J., Manning, C.D. and Heer, J. (2012) 'Termite: Visualization techniques for assessing textual topic models', in *Proceedings of the International Working Conference on Advanced Visual Interfaces (AVI '12)*. New York: ACM, pp. 74–77. doi: 10.1145/2254556.2254572.

Honnibal, M., Montani, I., Van Landeghem, S. and Boyd, A. (2020) *spaCy: Industrial-strength Natural Language Processing in Python*. Zenodo. doi: 10.5281/zenodo.1212303.

Johnson, R.B. and Onwuegbuzie, A.J. (2004) 'Mixed methods research: A research paradigm whose time has come', *Educational Researcher*, 33(7), pp. 14–26. doi: 10.3102/0013189X033007014.

Mihalcea, R. and Tarau, P. (2004) 'TextRank: Bringing order into texts', in *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*. Barcelona: Association for Computational Linguistics, pp. 404–411. Available at: https://aclanthology.org/W04-3252/ (Accessed: 10 October 2026).

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J. (2013) 'Distributed representations of words and phrases and their compositionality', *Advances in Neural Information Processing Systems*, 26, pp. 3111–3119. Available at: https://arxiv.org/abs/1310.4546 (Accessed: 10 October 2026).

Page, L., Brin, S., Motwani, R. and Winograd, T. (1999) *The PageRank citation ranking: Bringing order to the web*. Technical Report. Stanford InfoLab. Available at: http://ilpubs.stanford.edu:8090/422/ (Accessed: 10 October 2026).

Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: https://www.jmlr.org/papers/v12/pedregosa11a.html (Accessed: 10 October 2026).

Řehůřek, R. and Sojka, P. (2010) 'Software framework for topic modelling with large corpora', in *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. Valletta: ELRA, pp. 45–50. Available at: https://radimrehurek.com/gensim/ (Accessed: 10 October 2026).

Röder, M., Both, A. and Hinneburg, A. (2015) 'Exploring the space of topic coherence measures', in *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining (WSDM '15)*. New York: ACM, pp. 399–408. doi: 10.1145/2684822.2685324.

Sievert, C. and Shirley, K. (2014) 'LDAvis: A method for visualizing and interpreting topics', in *Proceedings of the Workshop on Interactive Language Learning, Visualization, and Interfaces*. Baltimore: Association for Computational Linguistics, pp. 63–70. Available at: https://aclanthology.org/W14-3110/ (Accessed: 10 October 2026).

Use of AI

I used AI (Claude) to help me understand the questions, write and debug the code step by step, interpret the outputs, draft and simplify the written explanations, and push the project to GitHub. I ran every cell myself and checked the outputs before writing up each answer.